

PRISM: Confidential Smart Contract Auditing with Pseudonymized Graph Screening

Anonymous Author(s)

Anonymous Institution

Abstract. Hosted language models make smart-contract audits disclose unreleased source code and business logic. We present PRISM, a confidential audit pipeline whose trust boundary is the audit host. PRISM strips secrets and reversibly pseudonymizes identifiers, turns Slither’s static single assignment (SSA) intermediate representation into typed function-level code property graphs (CPGs), screens and ranks functions with a two-stage graph attention model, audits the highest-ranked functions with a locally served open-weight code LLM, and validates reference harnesses with Foundry on a local EVM, restoring identifiers only in the final report. On 12,991 public function graphs covering five Smart Contract Weakness Classification (SWC) classes, the ranker reaches $75.95 \pm 0.19\%$ macro-F1 on a random split and $76.31 \pm 0.06\%$ on a lineage-disjoint split over five seeds, at 0.21 ms per function. Graph convolutional and graph isomorphism baselines reach the same level, share at least 97% of its errors and differ in none of 20 paired tests, indicating that contract-derived labels likely dominate the remaining error. On 57 contracts, pseudonymization preserves compilation and every Slither detector type, but a plaintext-frequency adversary re-links 89.5% of identifiers. Static filtering lowers safe contracts with alarms from 27/27 to 6/27 while retaining 18/30 vulnerable contracts; a five-case integrated trace completes every local stage and the two reference Foundry tests.

Keywords: Smart contract security · Confidential auditing · Graph neural networks · Local large language models · Invariant fuzzing · Pseudonymization.

1 Introduction

Smart contracts on the Ethereum Virtual Machine (EVM) hold large amounts of value in decentralized finance, and deployed bytecode cannot be patched in place [1, 2]. Flaws must therefore be found before deployment, usually by auditing source code that its owner has not yet released and considers proprietary [2].

Large language models (LLMs) are now the most active direction in automated auditing. GPTScan [3] and PropertyGPT [4] pair hosted GPT models with program analysis or formal verification, iAudit [5] combines fine-tuned detectors with LLM agents, LLM-SmartAudit [6] organizes multi-agent audits, and PromFuzz [7] and LLAMA [8] use LLMs to derive invariants or seeds for fuzzing.

Graph learning has advanced in parallel, from graph neural network (GNN) detectors over contract graphs [9, 10] to attention over local and global context [11] and heterogeneous graph transformers with LLM-derived features [12]. Classical tools explain why learning is attractive: 97% of 47,587 contracts were flagged by at least one static analyzer [13], and fuzzers need guidance to pass narrow guards [14, 15].

Most LLM auditors, however, call hosted models [3, 4, 6], so the auditor transmits unreleased code, identifier names and business logic to a model provider. Provider-side confidential inference remains a research prototype [16], and open-weight models fine-tuned for vulnerability detection [17] stop at a label without executable evidence. An auditor who must keep code in house therefore faces two open questions: which parts of an LLM-assisted audit can run on commodity local hardware while still producing executable evidence, and whether cheap pseudonymization of identifiers would make hosted models acceptable after all. Learned models already recover variable names from stripped binaries [18, 19], which suggests that renaming is fragile, but its residual exposure has not been measured for smart contract audits.

PRISM builds on the observation that the costly or risky steps of an LLM-assisted audit need different guarantees. Deciding *where* to look is a ranking problem that a compact graph model solves in well under a millisecond per function. Proposing *what* may be wrong benefits from a code LLM but does not need a frontier model when its output is treated as a hypothesis. Deciding *what is real* requires execution. PRISM therefore places a two-stage graph attention ranker in front of a locally served open-weight code LLM and a Foundry execution back end, and keeps every model and every artifact on the audit host; identifiers are pseudonymized before any learned stage and restored only in the final report. Our contributions are:

- We design PRISM, a confidential audit pipeline prototype with an explicit two-adversary threat model in which secret filtering, reversible pseudonymization, graph screening, hotspot-conditioned local LLM auditing, reference-harness validation and template repair all run on the audit host (Sect. 3). Among the systems we survey in Sect. 2, it is the only one that runs its learned stages locally, protects identifiers before model input and pairs screening with local reference-harness validation.
- We introduce a function-level code property graph (CPG) over Slither’s static single assignment (SSA) intermediate representation with typed control- and data-flow edges, and a two-stage graph attention ranker. We show over five seeds that it ranks functions at $76.31 \pm 0.06\%$ macro-F1 on unseen contract lineages in 0.21 ms per function, and that graph learners of different design converge to the same level and share at least 97% of their errors. An empirical structural-label-conflict diagnostic indicates that contract-derived labels, rather than encoder choice, dominate the remaining error (Sect. 5.1).
- We measure what reversible pseudonymization preserves and what it leaks. Masked code keeps compilation (57/57) and all 229 contract-level Slither de-

- tector types, whereas a frequency-aware adversary re-links 89.5% of identifiers, quantifying why masking must be paired with local inference (Sect. 5.3).
- We evaluate the local execution and filtering stages on a 57-contract benchmark: all 300 seeded Foundry runs on reference harnesses complete reproducibly, taxonomy- and severity-aware filtering reduces flagged safe contracts from 27/27 to 6/27 with six false alarms refuted in a case study while 18 statically flagged vulnerable contracts stay retained, and five template patches pass five validation checks (Sects. 5.2, 5.4 and 5.5).

The rest of the paper is organized as follows: Sect. 2 reviews related work, Sects. 3–4 present design and setup, Sect. 5 details results, and Sects. 6–7 discuss and conclude.

2 Related Work

Static analysis and fuzzing. Oyente [1] applies symbolic execution, Securify [20] checks Datalog patterns, and Slither [21] runs rule-based detectors on SlithIR, an SSA intermediate representation (IR); empirical studies and surveys [13, 22, 23] report low precision and uneven recall for such tools. Contract-Fuzzer [24], Echidna [25], sFuzz [26], Harvey [27], Smartian [14] and ItyFuzz [15] contribute call-level oracles, property-based, adaptive, greybox, data-flow-guided and snapshot-based fuzzing.

Graph learning. Surveys [28, 29] and models [9, 10] learn over contract graphs with normalized nodes and expert patterns, DA-GNN [30] applies dual attention, ConSense [11] combines local and global attention, and MANDO-LLM [12] feeds LLM code features into heterogeneous graph transformers. PRISM’s ranker builds on graph attention networks (GAT) and their GATv2 variant [31, 32] and operates on a homogeneous SSA-level function graph with typed edge attributes. Its role is to decide which functions later stages examine, not to issue the final verdict.

LLMs for auditing, fuzzing and repair. GPTScan [3] pairs GPT with static confirmation, PropertyGPT [4] and SmartInv [33] generate properties or invariants for verification, and LLM-SmartAudit [6] and iAudit [5] organize LLM agents around the audit. PromFuzz [7], LLAMA [8] and LLM4Fuzz [34] guide fuzzing, while Luo et al. [35] constrain contract generation. EVuLLM [17] fine-tunes small open models with QLoRA, while quantization (e.g., AWQ [36]) enables efficient local execution. On repair, surveys [37] formalize program repair, while sGuard [38] and Elysium [39] patch source or bytecode with templates.

Confidential analysis. Provider-side protection of prompts with confidential computing is an active topic [16]. On the attack side, ReSym [18] and GenNm [19] recover variable names and types from stripped binaries with fine-tuned LLMs, showing that removing or renaming identifiers does not remove the information

Table 1. Capability comparison with representative systems. GNN: graph detector; LLM: uses LLM; Local: on-premises model; Privacy: protects code before model input; Execution: dynamic confirmation (reference harnesses for PRISM); Repair: patch synthesis; n/a: not applicable; NR: not reported.

System	Year	GNN	LLM	Local	Privacy	Execution	Repair
Slither [21]	2019	✗	✗	n/a	n/a	✗	✗
sGuard [38]	2021	✗	✗	n/a	n/a	✗	✓
Smartian [14]	2021	✗	✗	n/a	n/a	✓	✗
Elysium [39]	2022	✗	✗	n/a	n/a	✗	✓
ItyFuzz [15]	2023	✗	✗	n/a	n/a	✓	✗
GPTScan [3]	2024	✗	✓	✗	✗	✗	✗
EVuLLM [17]	2025	✗	✓	✓	✗	✗	✗
PromFuzz [7]	2025	✗	✓	NR	✗	✓	✗
PropertyGPT [4]	2025	✗	✓	✗	✗	F	✗
LLAMA [8]	2026	✗	✓	NR	✗	✓	✗
MANDO-LLM [12]	2026	✓	✓	NR	✗	✗	✗
This work (PRISM)	2026	✓	✓	✓	✓	✓	✓

that structure carries. PRISM takes the conservative position: inference stays local, and pseudonymization protects only artifacts that leave the host.

Table 1 summarizes the capabilities. PRISM is not the strongest detector or fuzzer in isolation; its distinguishing property is the combination of local learned inference, identifier protection and reference-harness validation.

3 PRISM Design

Figure 1 shows the pipeline; no network access is required once the model weights are installed.

3.1 Threat Model and Pseudonymization

Assets and trust boundary. The asset is unreleased contract source code together with the business logic it reveals: identifier names, constants, addresses and endpoints. The trust boundary is the audit host operated by the contract owner or its auditor.

Adversaries. We consider (A1) third-party model providers and network observers, to which a cloud-LLM workflow would expose code, and (A2) honest-but-curious parties that later see PRISM artifacts outside the host, such as prompt logs, generated harnesses, bug reports shared with external reviewers, or a shared inference server inside the organization. PRISM removes A1 by construction, since no inference is remote, and DataMasker reduces what A2 learns from artifacts. A fully compromised audit host is out of scope, because such an adversary can read the plaintext source directly.

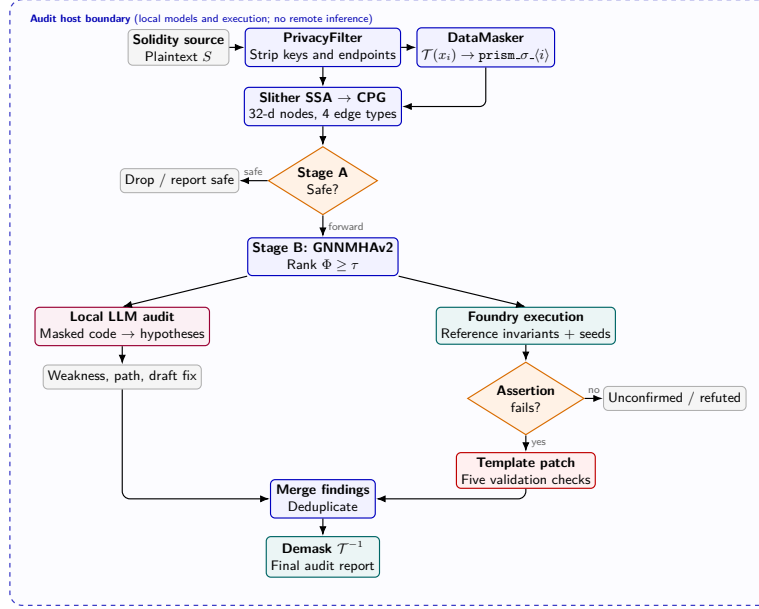


Fig. 1. PRISM architecture. All eight stages run inside the audit host. Ranked functions are audited by a local open-weight LLM, whose findings are hypotheses, and exercised by Foundry invariant harnesses, which confirm a finding only through a failing assertion. $\mathcal{T}$: pseudonymization map; Φ : ranking score; τ : forwarding threshold.

Mechanism. *PrivacyFilter* first removes hard-coded private keys and application programming interface (API) keys, remote procedure call (RPC) endpoints and 20-byte addresses. *DataMasker* then renames user-defined identifiers of contracts, types, functions and variables, and leaves Solidity keywords, EVM built-ins such as `msg.sender` and `block.timestamp`, standard interfaces such as `IERC20`, numeric literals and comments untouched. The i -th distinct identifier x_i is mapped to

$$\mathcal{T}(x_i) = \text{prism_}\sigma(x_i)_ \langle i \rangle, \quad (1)$$

where $\langle i \rangle$ is a four-digit counter and $\sigma(x) \in \{\text{Type}, \text{b}, \text{v}\}$ keeps a coarse lexical role: **Type** for capitalized names, **b** for boolean-style names beginning with *is*, *has* or *can*, and **v** otherwise. $\mathcal{T}$ is a bijection; its inverse $\mathcal{T}^{-1}$ never leaves the host and is applied only to the final report. Sect. 5.3 measures what $\mathcal{T}$ preserves and what it still reveals to A2.

3.2 Function-Level Code Property Graph

From the pseudonymized source S_{mask} , PRISM extracts Slither’s SSA IR [21] and builds one directed graph per function, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X, E_{\text{attr}})$, whose nodes are SSA instructions.

- **Node features** $X \in \mathbb{R}^{N \times 32}$: a 13-dimensional one-hot instruction type, 10 security indicators (low-level calls, storage reads and writes, guards, transaction-context reads, etc.), and 9 structural attributes (in- and out-degree, loop membership, relative depth, etc.).
- **Edge attributes** $E_{attr} \in \{0, 1\}^{|\mathcal{E}| \times 4}$: a one-hot edge type among **sequential**, **branch**, **loop_back** (control flow) and **data_dep** (def-use), so that control and data flow share one edge set.

3.3 Two-Stage Graph Attention Ranker

Both stages use the same encoder, GNNMHAv2, which projects node features to $d = 256$ dimensions and stacks $L = 4$ edge-aware graph attention layers with $K = 8$ heads [31, 32]. With edge attribute $\mathbf{e}_{ij}$ and $\mathcal{N}^+(i) = \mathcal{N}(i) \cup \{i\}$, head k of layer l computes

$$\alpha_{ij}^{k,l} = \text{softmax}_{j \in \mathcal{N}^+(i)} \left(\mathbf{a}^{k,l\top} \text{LeakyReLU}(W_s^{k,l} h_i^l + W_t^{k,l} h_j^l + W_e^{k,l} \mathbf{e}_{ij}) \right), \quad (2)$$

$$h_i^{l+1} = \text{ELU} \left(\text{BN} \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}^+(i)} \alpha_{ij}^{k,l} W^{k,l} h_j^l \right) \right) + h_i^l, \quad (3)$$

where BN is batch normalization and ELU the exponential linear unit. Residual connections and a jumping-knowledge (JK) readout, $z_i = [h_i^1 \| h_i^2 \| h_i^3 \| h_i^4] \in \mathbb{R}^{1024}$, keep both local and multi-hop patterns. The graph embedding $g \in \mathbb{R}^{2048}$ concatenates attention pooling and mean pooling,

$$g = \left[\sum_i \beta_i z_i \parallel \frac{1}{N} \sum_i z_i \right], \quad \beta_i = \text{softmax}_i (\mathbf{w}^\top \text{ReLU}(W_p z_i)), \quad (4)$$

and a two-hidden-layer perceptron maps g to class logits. One layer costs $O(K(|\mathcal{V}|d^2 + |\mathcal{E}|d))$, which is linear in graph size for fixed d .

Stage A is a binary model (vulnerable or safe) trained with class-weighted, label-smoothed cross-entropy (CE). Stage B is a five-class model trained with class-weighted focal loss on label-smoothed CE, with smoothing $\epsilon = 0.05$ and $\gamma = 2$:

$$\mathcal{L} = \frac{1}{n} \sum_{m=1}^n \alpha_{y_m} (1 - p_m)^\gamma \text{CE}_\epsilon(g_m, y_m), \quad (5)$$

where p_m is the predicted probability of the true class and α_y the class weight. At inference, Stage A drops functions it predicts safe, Stage B assigns a class to the remaining ones, and PRISM ranks them by the Stage-B confidence $\Phi(\mathcal{G}) = \max_c p_B(c | \mathcal{G})$, forwarding functions with $\Phi \geq \tau$ ($\tau = 0.3$) in decreasing order of Φ .

3.4 Local LLM Audit, EVM Execution and Repair

Local LLM audit. The masked source and the ranked functions, each with its predicted class and Φ , are passed to an open-weight code LLM, DeepSeek-Coder-V2-Lite-Instruct, a 16B-parameter mixture-of-experts model with 2.4B

active parameters [40]. It is served on the host by Ollama¹ with temperature 0.1 and at most 4,096 generated tokens. The prompt asks for a structured finding per suspected weakness: weakness type, severity, affected function, attack path, exploit scenario and fix. PRISM treats these findings as hypotheses.

Execution. To verify findings dynamically without network calls, PRISM interfaces with a local Foundry execution back end. For evaluated classes, invariant harnesses are instantiated from class templates (e.g., balance conservation for reentrancy, privileged rejection for access control) and fuzzed locally. We benchmark execution using canonical reference harnesses (Sect. 5.2) and an empirical safe-case replay (Sect. 5.4); automated harness synthesis across arbitrary contracts remains an open challenge (Sect. 6). A finding is confirmed only if an assertion fails with a concrete counterexample trace.

Repair and report. A template-based repair module proposes a patch for each finding, which must pass five checks (Sect. 5.5). LLM hypotheses and execution results are merged and deduplicated by function and weakness type, and identifiers are restored through $\mathcal{T}^{-1}$ only in the final report.

4 Experimental Setup

Research questions (RQs). **RQ1:** How accurately does the ranker classify vulnerable functions, on seen and unseen contract lineages, and how does it compare with other graph learners? **RQ2:** Does the local EVM back end execute harnesses reliably and reproducibly? **RQ3:** Does pseudonymization preserve analysis utility, and what does it still reveal? **RQ4:** How far do taxonomy- and severity-aware filters reduce alarms on safe contracts without losing vulnerable ones? **RQ5:** Do template patches block exploits while preserving interfaces?

Dataset. We extract 12,991 function-level CPGs with at least three nodes from three public sources: SmartBugs Wild [13] (1,179 graphs), the SolidiFI benchmark [22] (8,105 graphs) and LISA-Bench² (3,707 graphs), a collection of professional audit findings. The graphs cover five classes of the Smart Contract Weakness Classification (SWC) registry: Reentrancy (SWC-107, 2,475), Integer Overflow (SWC-101, 2,392), Access Control (SWC-105, 2,026), Unchecked Return Value (SWC-104, 2,983) and Timestamp / Transaction-Order Dependence (SWC-114, 3,115). Graphs of the remaining source categories (denial of service, delegatecall) are excluded from the five-class task. Labels follow a contract-derived weak-supervision policy: SmartBugs and SolidiFI functions inherit the label of their contract, and LISA-Bench functions inherit the label obtained by keyword mapping of the finding text. We audited this choice on SolidiFI: of 9,369 injected bug spans in its 350 BugLog contracts, 4,947 map uniquely to a function body, and a strictly span-mapped subset would keep only 2,580 graphs with two

¹ <https://ollama.com>; the quantized model occupies 8.3 GiB on disk.

² AgentLISA: LISA-Bench: Smart contract vulnerability detection benchmark. <https://github.com/agentlisa/bench> (2025), accessed September 2026.

Table 2. Comparison with graph baselines on both splits (five-seed mean $\pm$ std). Single-seed GAT and GATv2 score within 0.5 points of GNNMHA_v2 (0.7562 and 0.7571). Slither is mapped to the five classes. Best mean per column in bold.

Method	Random split ($n = 1,949$)		Lineage-disjoint split ($n = 2,146$)	
	Accuracy	Macro-F1	Accuracy	Macro-F1
Slither 0.11.5 (mapped)	0.2299	0.0748	0.2330	0.0756
GCN (5 seeds)	0.7434 ± 0.0016	0.7604 ± 0.0022	0.7445 ± 0.0008	0.7630 ± 0.0009
GIN (5 seeds)	0.7442 ± 0.0005	0.7609 ± 0.0005	0.7432 ± 0.0011	0.7617 ± 0.0011
GNNMHA _v 2 (PRISM, 5 seeds)	0.7426 ± 0.0014	0.7595 ± 0.0019	0.7445 ± 0.0006	0.7631 ± 0.0006

Reentrancy and two timestamp dependence examples, too few for a five-class task. The resulting label noise is discussed in Sect. 6. Stage A uses all 24,454 graphs as a binary task (labels 0–7 vulnerable; 8 safe).

Splits. We use two frozen protocols for the five-class ranker: (i) *Random split*: stratified 70/15/15 (9,093/1,949/1,949 graphs); and (ii) *lineage-disjoint split*: contracts are partitioned by origin so that SolidiFI `buggy_N` variants, functions from the same LISA-Bench finding, and functions from the same imported source contract never cross partitions, giving 8,349/2,496/2,146 graphs with zero lineage overlap. Stage A instead uses a frozen binary 70/15/15 split (17,118/3,667/3,669; test: 3,132 vulnerable, 537 safe).

Baselines. To isolate the effect of the encoder, we train graph convolutional (GCN) and graph isomorphism (GIN) networks as well as GAT and GATv2 on the same graphs, node features, splits and training budget; GAT and GATv2 do not use edge attributes. As a rule-based reference, Slither 0.11.5 is mapped to the five classes through its detector names (e.g., `reentrancy-eth` to Reentrancy, `unchecked-lowlevel` to Unchecked Return), and a function that triggers no mapped detector is assigned the majority training class.

Implementation. Models are implemented in PyTorch Geometric and trained with AdamW (learning rate 10^{-3} , weight decay 10^{-4}), batch size 64 and OneCycleLR for up to 35 epochs with early stopping on validation macro-F1 (patience 15). GNNMHA_v2, GCN and GIN are trained with five seeds (42 to 46); GAT and GATv2 with seed 42. Experiments run on an Apple M4 CPU (10 cores, 16 GB RAM). One 35-epoch training run of GNNMHA_v2 takes 348.3 ± 37.2 s, compared with 56.5 ± 6.7 s for GCN and 67.4 ± 1.3 s for GIN, and inference takes 0.21 ms per function graph. Execution uses Foundry Forge 1.5.1 and `solc` 0.8.30.

5 Results

5.1 RQ1: Function-Level Ranking Accuracy

Table 2 compares the ranker with graph baselines. GNNMHA_v2 reaches 0.7595 ± 0.0019 macro-F1 on the random split and 0.7631 ± 0.0006 on the lineage-

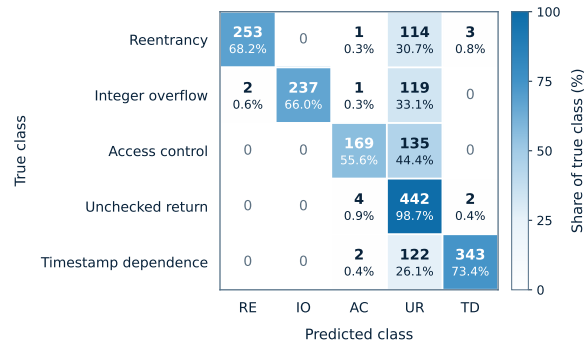


Fig. 2. Confusion matrix of a single GNNMHA v2 training run on the frozen random split (seed 42; $n = 1,949$; accuracy 0.7414, macro-F1 0.7581; five-seed results are in Table 2). Cells show counts and the share of the true class. Columns follow the row order: Reentrancy (RE, SWC-107), Integer overflow (IO, SWC-101), Access control (AC, SWC-105), Unchecked return (UR, SWC-104), Timestamp dependence (TD, SWC-114).

disjoint split, with macro-precision 0.887 ± 0.005 and macro-recall 0.725 ± 0.001 on the random split. The mapped Slither reference scores 0.0748 because its detectors fire on few function slices under our label mapping. Over five fresh seeds, Stage A reaches $88.62 \pm 0.24\%$ accuracy ($87.82 \pm 0.32\%$ vulnerable recall; $93.26 \pm 0.61\%$ safe specificity; $n = 3,669$).

The more informative result is that all graph learners converge to the same level. On both splits, GCN, GIN, GAT, GATv2 and GNNMHA v2 lie within 0.5 macro-F1 points of each other. In 20 paired exact McNemar tests between GNNMHA v2 and GCN or GIN (two splits, five seeds), no difference is significant (all $p > 0.16$), the paired models disagree on at most 22 test functions per run, and between 97.4% and 99.6% of GNNMHA v2’s errors are shared with the baseline. Pairwise Jaccard error overlap exceeds 96.5%, and 493 of 520 errors (94.8%) are shared across all five architectures. When diverse models make identical errors, the result is consistent with label noise being a stronger limitation than encoder choice. A structural-label-conflict diagnostic confirms this: 6.2% of random-split test graphs match a training graph with a conflicting label due to contract-level inheritance; on these conflicting graphs, accuracy collapses to 7.9%, whereas on conflict-free graphs it reaches 78.7%.

The per-class view in Fig. 2 shows where the shared errors go. Precision is 0.95 to 1.00 for the four specific classes, but their recall is 0.56 to 0.73, and 490 of the 505 errors are predictions of *unchecked return* (precision 0.47, recall 0.99), whose SSA signature, an external call followed by an unused result, is the most generic. The model therefore rarely confuses two specific weaknesses. It assigns functions that carry no class-specific evidence to the most generic class, which is what contract-derived labels produce when a contract’s label is inherited by functions that do not contain the flaw. Consistently, top-2 accuracy (five-seed

Table 3. Ablation of GNNMHA_{v2} on the random split (five seeds, mean $\pm$ std). Δ : change in mean macro-F1 relative to the full model.

Variant	Macro-F1	Δ	Top-2 accuracy
Full model	0.7595 ± 0.0019	–	0.8086
without edge attributes	0.7603 ± 0.0011	+0.0008	0.8085
without jumping knowledge	0.7597 ± 0.0018	+0.0002	0.8117
focal loss $\rightarrow$ cross-entropy	0.7584 ± 0.0012	–0.0010	0.8097
attention+mean $\rightarrow$ mean pooling	0.7596 ± 0.0029	+0.0001	0.8115
$L = 2$	0.7611 ± 0.0016	+0.0016	0.8079
$L = 3$	0.7588 ± 0.0025	–0.0007	0.8069
$K = 4$	0.7588 ± 0.0017	–0.0007	0.8089

mean) is 0.8086, so the second-ranked class recovers only 25.6% of the errors: most errors are not near misses.

The ablation in Table 3 supports the same reading. Removing edge attributes, jumping knowledge or attention pooling, replacing focal loss, and changing depth or head count all move macro-F1 by at most 0.16 points, within one to two standard deviations of seed variation. Under contract-derived labels, encoder design is not the bottleneck; label granularity is. For PRISM this has a practical consequence. The ranker’s job is to order functions for later stages, any of these encoders does so at the same quality, and at 0.21 ms per function the choice costs nothing next to execution. Better labels, not a larger encoder, are the lever for better ranking.

5.2 RQ2: Execution Back-End Reliability

We executed expert-written reference harnesses for five case-study contracts under 30 fuzzing seeds (101 to 130) in a directed exploit test and a 10,000-call fuzzing campaign, for $5 \times 30 \times 2 = 300$ runs. The targets cover the taxonomy: SimpleDAO (SWC-107, reentrancy), TokenSale (SWC-101, integer overflow), UnprotectedVault (SWC-105, access control), UncheckedBank (SWC-104, unchecked return), and TimestampLock (SWC-114, timestamp dependence). All 300/300 runs passed without timeouts (mean directed/fuzz latency 190.26/202.75 ms); directed transaction gas was deterministic at 77,558, 39,648, 48,550, 27,202 and 39,827, respectively. A seed-42 integrated trace ran masking, preprocessing, GNN routing, bounded local LLM audit and reference tests for all five cases; every stage completed and 5/5 directed and 10,000-call tests passed. It demonstrates orchestration, not automated harness generation or benchmark-wide end-to-end performance. Because the harnesses are hand-written, RQ2 does not measure generated-harness bug-finding frequency (Sect. 6).

5.3 RQ3: Pseudonymization Fidelity and Residual Exposure

Utility. Table 4 shows that DataMasker is lossless for compilation and restoration: every masked contract compiles with `solc`, and $\mathcal{T}^{-1}$ restores every contract

Table 4. DataMasker on the 57 benchmark contracts (286 user-defined identifiers). Upper block: utility (higher is better). Lower block: exposure to an A2 adversary (lower is better).

Metric	Value	Meaning
Compilation validity	57/57 (100%)	masked code compiles
Exact restoration	57/57 (100%)	$\mathcal{T}^{-1}$ restores the source
Slither detector types retained	229/229 (100%)	per contract; 6 added
Dictionary exposure	87/286 (30.4%)	name in a 32-term domain list
Frequency re-linkage	256/286 (89.5%)	rank matching of counts

exactly. For static analysis, all 229 contract-level Slither detector types raised on plaintext code are preserved on masked code (235 total; six added by naming conventions). Under taxonomy-guided auditing across the five SWC classes, raw-code recall on the 30 vulnerable contracts reaches 30/30 (100.0%) with 11/27 (40.7%) safe specificity, showing that unconstrained LLM inspection raises substantial false alarms on benign patterns. On pseudonymized code, masked recall achieves 13/30 (43.3%) while safe specificity increases to 19/27 (70.4%). This unguided text-only baseline drops recall when role-revealing names (e.g., `owner`) are masked, demonstrating why PRISM couples local LLMs with structural GNN screening to isolate suspicious slices without relying on identifier semantics. Crucially, across all contracts with EVM built-ins (`block.timestamp`, external `.call`), raw and masked hypotheses achieve identical predictions ($J = 1.00$).

Exposure. We quantify two A2 adversaries. The *dictionary* measure counts identifiers whose plaintext name is one of 32 common Solidity and DeFi terms (e.g., `owner`, `balance`, `withdraw`); an adversary who guesses these terms from the role of a pseudonym recovers at most 30.4% of identifiers. The *frequency* adversary knows how often each plaintext identifier occurs in the contract, for example from an earlier version or a leaked excerpt, ranks identifiers and pseudonyms by occurrence count, and aligns the two rankings. Because $\mathcal{T}$ renames one-to-one and preserves occurrence counts, only ties stop it, and it re-links 89.5% of identifiers. This is an upper bound under strong side knowledge, and it is the informative result: renaming preserves the structure of the code, so pseudonymization alone is not a confidentiality mechanism for code sent to an untrusted model, in line with learned name recovery from binaries [18, 19]. It is defense in depth for artifacts that leave the host, and this measurement is why PRISM keeps inference local instead of masking code and sending it to a hosted LLM.

5.4 RQ4: Alarm Reduction on Safe Contracts

We ran Slither 0.11.5 on the 57-contract benchmark (27 safe and 30 vulnerable contracts, labeled by their benchmark templates) and applied PRISM’s static filters in sequence (Table 5).

Unfiltered Slither raises 230 alarms and flags every contract, yielding 0/27 (0.0%) specificity and 30/57 (52.6%) precision due to stylistic detectors. Restrict-

Table 5. Alarms along the *static* filtering cascade on 57 contracts. “Safe cleared” measures contract-level specificity; “Vuln. retained” tracks preservation of Slither-flagged vulnerable contracts.

Stage	Filter	Alarms	Safe cleared (↑)	Vuln. retained
0	Slither, all detectors	230	0/27 (0.0%)	30/30 (100%)
1	Five target SWC classes only	41	21/27 (77.8%)	18/30 (60.0%)
2	High/medium impact, confidence	33	21/27 (77.8%)	18/30 (60.0%)

ing output to the five target classes (Stage 1) clears 21 of 27 safe contracts (77.8% specificity); 12 vulnerable contracts lack dedicated Slither detectors under this mapping. Filtering for high/medium impact and confidence (Stage 2) eliminates eight secondary/low-severity alarms on already-flagged contracts, preserving contract-level specificity (21/27) while retaining all 18 detected flaws. Separately, template-based execution validation on 57 labeled benchmark fixtures (20.90 s in Foundry under `solc` 0.8.30) ran deterministically: 30 labeled vulnerable fixtures satisfy exploit predicates and 27 safe fixtures satisfy safety predicates, confirming concrete EVM feasibility while automated synthesis for arbitrary mainnet test suites remains future work.

5.5 RQ5: Patch Validation

For the five case-study contracts with exploit harnesses (Sect. 5.2), template repair produced one patch each. Each patch passed five checks: (1) compilation with `solc` 0.8.30; (2) minimal change ($\leq 25\%$ of lines via an AST parse); (3) the original exploit harness now reverts; (4) the 10,000-call fuzzing campaign still passes; and (5) function selectors and storage layout are unchanged, preserving the external ABI and proxy storage layout.

6 Discussion and Threats to Validity

Graph screening is linear in graph size, costing 0.21 ms per function after CPG extraction (negligible next to one Foundry run of 0.19–0.20 s, Sect. 5.2); the LLM stage is invoked only for functions with $\Phi \geq \tau$, bounding generations per contract.

Threats to validity. *Construct.* Function graphs inherit contract-derived labels, so benign functions are counted as positives, bounding ranking accuracy. The 57 benchmark labels follow injection templates rather than manual spans, and SWC-114 targets are timestamp proxies. *Internal.* The 57 contracts follow standardized invariant and exploit templates; automated harness synthesis for arbitrary multi-file mainnet contracts remains open work. Slither IR extraction may degrade on inline Yul. DataMasker preserves string literals and comments. *External.* Direct cross-dataset evaluation on LISA-Bench without fine-tuning

dropped macro-F1 from 0.8985 to 0.5157. Benchmark contracts are small reference implementations under `solc` 0.8.30.

Data availability. Code, configurations, frozen split manifests, result files and reference harnesses are available in an anonymized repository: <https://anonymous.4open.science/r/PRISM-PROJECT/>.

7 Conclusion

This paper presented PRISM, a confidential smart contract audit pipeline keeping every model and artifact on the audit host. Its two-stage graph attention ranker orders functions at $76.31 \pm 0.06\%$ macro-F1 on unseen lineages in 0.21 ms per function, showing that contract-derived labels dominate error rather than model capacity. Reversible pseudonymization preserves compilation, restoration, and static analysis, while an 89.5% frequency re-linkage demonstrates that local inference is essential. Together with reproducible EVM execution, static filtering reducing flagged safe contracts from 27/27 to 6/27, and reference-harness validation on scoped case studies, PRISM makes confidential auditing practical for teams that cannot share code. Future work will expand end-to-end evaluation on larger production benchmarks, replace contract-derived supervision with precise function-level ground truth, and extend template repair to automated multi-harness patch synthesis.

References

1. Luu, L., Chu, D.H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. pp. 254–269 (2016)
2. Atzei, N., Bartoletti, M., Cimoli, T.: A survey of attacks on ethereum smart contracts (sok). In: International conference on principles of security and trust. pp. 164–186. Springer (2017)
3. Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th international conference on software engineering. pp. 1–13 (2024)
4. Liu, Y., Xue, Y., Wu, D., Sun, Y., Li, Y., Shi, M., Liu, Y.: Propertygpt: Llm-driven formal verification of smart contracts through retrieval-augmented property generation. arXiv preprint arXiv:2405.02580 (2024)
5. Ma, W., Wu, D., Sun, Y., Wang, T., Liu, S., Zhang, J., Xue, Y., Liu, Y.: Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). pp. 1742–1754. IEEE (2025)
6. Wei, Z., Sun, J., Zhang, Z., Zhang, X., Li, M., Hou, Z.: Llm-smartaudit: Advanced smart contract vulnerability detection. arXiv preprint arXiv:2410.09381 (2024)
7. Lin, X., Xie, Q., Zhao, B., Tian, Y., Zonouz, S., Ruan, N., Li, J., Beyah, R., Ji, S.: Promfuzz: Leveraging llm-driven and bug-oriented composite analysis for detecting functional bugs in smart contracts. In: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE). pp. 1044–1056. IEEE (2025)

8. Gai, K., Liang, H., Yu, J., Zhu, L., Niyato, D.: Llama: Multi-feedback smart contract fuzzing framework with llm-guided seed generation. *IEEE Transactions on Information Forensics and Security* (2026)
9. Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q.: Smart contract vulnerability detection using graph neural network. In: *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*. pp. 3283–3290 (2020)
10. Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X.: Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Transactions on Knowledge and Data Engineering* **35**(2), 1296–1310 (2021)
11. Pang, Y., Liu, X., Huang, T., Hong, Y., Huang, J., Duan, S., Dong, C.: Graph-based contract sensing framework for smart contract vulnerability detection. *IEEE Transactions on Big Data* (2025)
12. Nguyen, N.M., Nguyen, H.H., Long Le, T., Ahmadi, Z., Doan, T.N., Wu, D., Jiang, L.: Mando-llm: Heterogeneous graph transformers with large language models for smart contract vulnerability detection. *ACM Transactions on Software Engineering and Methodology* **35**(6), 1–30 (2026)
13. Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: *Proceedings of the ACM/IEEE 42nd International conference on software engineering*. pp. 530–541 (2020)
14. Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K.: Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. pp. 227–239. IEEE (2021)
15. Shou, C., Tan, S., Sen, K.: Ityfuzz: Snapshot-based fuzzer for smart contract. In: *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. pp. 322–333 (2023)
16. Li, C., Gim, I., Zhong, L.: Confidential prompting: Privacy-preserving llm inference on cloud. *arXiv preprint arXiv:2409.19134* (2024)
17. Mandana, E., Vlahavas, G., Vakali, A.: Evullm: ethereum smart contract vulnerability detection using large language models. *Electronics* **14**(16), 3226 (2025)
18. Xie, D., Zhang, Z., Jiang, N., Xu, X., Tan, L., Zhang, X.: Resym: Harnessing llms to recover variable and data structure symbols from stripped binaries. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. pp. 4554–4568 (2024)
19. Xu, X., Zhang, Z., Su, Z., Huang, Z., Feng, S., Ye, Y., Jiang, N., Xie, D., Cheng, S., Tan, L., Zhang, X.: Unleashing the power of generative model in recovering variable names from stripped binary. In: *Network and Distributed System Security Symposium (NDSS)* (2025)
20. Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. pp. 67–82 (2018)
21. Feist, J., Grieco, G., Groce, A.: Slither: a static analysis framework for smart contracts. In: *2019 IEEE/ACM 2nd international workshop on emerging trends in software engineering for blockchain (WETSEB)*. pp. 8–15. IEEE (2019)
22. Ghaleb, A., Pattabiraman, K.: How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In: *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. pp. 415–427 (2020)
23. Zhu, H., Yang, L., Wang, L., Sheng, V.S.: A survey on security analysis methods of smart contracts. *IEEE Transactions on Services Computing* **17**(6), 4522–4539 (2024)

24. Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE international conference on automated software engineering. pp. 259–269 (2018)
25. Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis. pp. 557–560 (2020)
26. Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In: Proceedings of the ACM/IEEE 42nd international conference on software engineering. pp. 778–788 (2020)
27. Wustholz, V., Christakis, M.: Harvey: a greybox fuzzer for smart contracts. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1398–1409 (2020)
28. Alsunaidi, S.J., Aljamaan, H., Hammoudeh, M.: Leveraging machine learning models to improve smart contract security: A survey of vulnerabilities and detection methods. *ACM Computing Surveys* **58**(6), 1–37 (2025)
29. Wang, S., Huang, J., Chen, Z., Song, Y., Tang, W., Mao, H., Fan, W., Liu, H., Liu, X., Yin, D., et al.: Graph machine learning in the era of large language models (LLMs). *ACM Transactions on Intelligent Systems and Technology* **16**(5), 1–40 (2025)
30. Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H.: Da-gnn: a smart contract vulnerability detection method based on dual attention graph neural network. *Computer Networks* **242**, 110238 (2024)
31. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017)
32. Brody, S., Alon, U., Yahav, E.: How attentive are graph attention networks? *arXiv preprint arXiv:2105.14491* (2021)
33. Wang, S.J., Pei, K., Yang, J.: Smartinv: Multimodal learning for smart contract invariant inference. In: 2024 IEEE Symposium on Security and Privacy (SP). pp. 2217–2235. IEEE (2024)
34. Shou, C., Liu, J., Lu, D., Sen, K.: Llm4fuzz: Guided fuzzing of smart contracts with large language models. *arXiv preprint arXiv:2401.11108* (2024)
35. Luo, H., Lin, Y., Yan, X., Hu, X., Wang, Y., Zeng, Q., Wang, H., Jiang, J.: Guiding LLM-based smart contract generation with finite state machine. In: Proceedings of the 34th International Joint Conference on Artificial Intelligence (IJCAI) (2025)
36. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S.: AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In: Proceedings of Machine Learning and Systems (MLSys). vol. 6, pp. 87–100 (2024)
37. Yang, B., Cai, Z., Liu, F., Le, B., Zhang, L., Bissyandé, T.F., Liu, Y., Tian, H.: A survey of llm-based automated program repair: Taxonomies, design paradigms, and applications. *arXiv preprint arXiv:2506.23749* (2025)
38. Nguyen, T.D., Pham, L.H., Sun, J.: Sguard: Towards fixing vulnerable smart contracts automatically. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1215–1229. IEEE (2021)
39. Ferreira Torres, C., Jonker, H., State, R.: Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses. pp. 115–128 (2022)
40. Zhu, Q., Guo, D., Shao, Z., Yang, D., Wang, P., Xu, R., Wu, Y., Li, Y., Gao, H., Ma, S., et al.: Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024)